

Seguridad en IA para MLOps

Proteger, operar y gobernar la plataforma. Aplicar SAIF a cada capa que construimos: REST, GraphQL, gRPC, streaming, federado y data lake.

Framework SAIF

Amenazas + defensas

Operación y gobernanza

Sesión 7 · último Mini-TP individual (cierra Mini-TPs 1–7)

Agenda

- 1 Por qué importa** el dilema oportunidad vs. riesgo · riesgos en la empresa
- 2 Conocer el sistema de IA** superficie de ataque · amenazas específicas de IA
- 3 SAIF** qué es, sus 6 elementos y su mapa de riesgos
- 4 Defensa en profundidad** las capas de seguridad sobre nuestra plataforma
- 5 SAIF por cada capa vista** REST · GraphQL · gRPC · streaming · federado · data lake
- 6 Operación y gobernanza** observabilidad, drift, model cards, auditoría, CI/CD
- 7 Práctica + Mini-TP 7** integrador + un notebook guiado por protocolo

¿Dónde estamos?

Construimos una plataforma completa: el modelo se sirve, los datos fluyen y se almacenan. Hoy la pregunta cambia: ¿cómo la protegemos, operamos y gobernamos?

Lo que ya construimos

- REST · GraphQL · gRPC — el modelo servido por varios protocolos
- Streaming — inferencia sobre eventos en tiempo real
- Federado — entrenamiento sin centralizar datos
- Data Lake — datos, artefactos y modelos en MinIO/S3

Lo que agregamos hoy

- **Cada capa tiene su propia superficie de ataque.**
- SAIF: un marco para asegurar todo el sistema de IA.
- Defensas concretas: autenticación, validación, cifrado...
- Operar y gobernar: monitoreo, auditoría, model cards.

REST

GraphQL

gRPC

Streaming

Federado

Cloud/Lake

Seguridad

Integra

¿Por qué importa la seguridad de la IA?

El dilema: oportunidad vs. riesgo

La IA está en todas partes y la presión por adoptarla es enorme. Pero cada modelo en producción es también una nueva superficie de ataque. La pregunta de negocio: ¿es segura para mi organización?

Lo que está en juego

- Fuga de datos sensibles (clientes, negocio).
- Decisiones manipuladas por un atacante.
- Robo del modelo (propiedad intelectual).
- Incumplimiento legal y pérdida de confianza.

Seguridad no es una capa que se agrega al final: es una propiedad de todo el ciclo de vida del modelo — de los datos al despliegue, la operación y la gobernanza.

Conocer el sistema de IA = asegurarlo

No se puede proteger lo que no se conoce. Un sistema de IA tiene cuatro componentes, y cada uno es una puerta posible para un atacante.

API / Interfaz

Donde usuarios y sistemas envían entradas y reciben resultados. *Riesgo: Abuso, inyección, evasión, DoS, scraping del modelo*

Ingesta y almacenamiento

El flujo de datos y dónde se guardan (el data lake). *Riesgo: Envenenamiento, acceso no autorizado, fuga*

Modelo

El artefacto entrenado y servido. *Riesgo: Robo/extracción, inversión, manipulación del artefacto*

Entrenamiento / datos

El pipeline que produce el modelo. *Riesgo: Datos envenenados, backdoors, dependencias inseguras*

Amenazas específicas de la IA

Envenenamiento de datos

El atacante corrompe el dato de entrenamiento para degradar o sesgar el modelo (o instalar un backdoor).

Ataque adversario (evasión)

Entradas perturbadas a propósito para forzar una predicción errónea en inferencia.

Extracción / robo del modelo

Consultar masivamente la API para clonar el modelo o recuperar sus parámetros.

Inversión / fuga de privacidad

Inferir datos de entrenamiento sensibles a partir de las salidas del modelo.

Manipulación del artefacto

Alterar el modelo o su despliegue (supply chain del modelo).

Abuso e inyección

Prompt/entrada maliciosa, DoS, o abuso de la integración con otros sistemas.

SAIF — Secure AI Framework

SAIF (Google) es un marco práctico para llevar la seguridad a todo el ciclo de vida de la IA. Seis elementos guían el trabajo:

1 • Bases sólidas

Extender los fundamentos de seguridad al ecosistema de IA (identidades, secretos, cifrado).

2 • Detectar y actuar

Incorporar la IA al monitoreo y la respuesta a incidentes de la organización.

3 • Automatizar defensas

Usar herramientas que escalen al ritmo de las amenazas, nuevas y existentes.

4 • Coherencia de controles

Controles a nivel de plataforma, consistentes en todos los sistemas.

5 • Mejora continua

Adaptar las mitigaciones y cerrar ciclos de feedback rápidos (red teaming, reentrenar).

6 • Contextualizar el riesgo

Situar los riesgos del sistema de IA en el proceso de negocio que lo rodea.

El mapa de riesgos de SAIF

SAIF ubica los riesgos sobre los componentes del sistema. Así se decide QUÉ control aplicar y DÓNDE.

Componente	Riesgo principal (SAIF)	Control
Datos	Envenenamiento, manejo excesivo de datos	Validación, control de acceso, linaje
Infraestructura	Manipulación del despliegue, acceso no autorizado	Identidad, secretos, aislamiento, cifrado
Modelo	Exfiltración, ingeniería inversa, tampering	Control de acceso al artefacto, versionado, firma
Aplicación	Inyección, evasión, salida insegura	Validación de entrada/salida, rate limiting

Defensa en profundidad sobre la plataforma

Ninguna medida basta sola. Se apilan capas: si una falla, la siguiente contiene el daño. Así se ven sobre nuestra plataforma MLOps.

Autenticación y autorización — quién entra y qué puede hacer (API key, JWT, roles)

Validación de entrada/salida + rate limiting — frenar inyección, evasión y DoS

Secretos y configuración — credenciales fuera del código (.env, vault)

Datos y modelo — cifrado, versionado/integridad, acceso mínimo al lake

Entrenamiento robusto — adversarial training, privacidad diferencial, agregación robusta

Operación y gobernanza — monitoreo, auditoría, model cards, respuesta a incidentes

Capa 1 — Autenticación y autorización

Ideas

- Autenticación: verificar la identidad (API key, token JWT).
- Autorización: qué puede hacer cada identidad (roles/alcances).
- **Nunca exponer el modelo sin control de acceso.**
- JWT: firma verificable, con expiración; no guardar secretos en el token.
- Mismo principio para REST, GraphQL y gRPC.

```
# FastAPI: API key como dependencia

from fastapi import FastAPI, Depends, HTTPException
from fastapi.security import APIKeyHeader

api_key = APIKeyHeader(name="X-API-Key")

def auth(k: str = Depends(api_key)):
    if k != CLAVE_VALIDA:
        raise HTTPException(401, "no autorizado")

@app.post("/predict", dependencies=[Depends(auth)])
def predict(x: Entrada):
    ...
```

Capa 2 — Validación y límites

Qué frena cada control

- Validar la entrada (tipos, rangos, tamaño): frena inyección y entradas raras.
- Validar/limitar la salida: no filtrar internals ni datos sensibles.
- **Rate limiting: frena DoS y el scraping masivo para robar el modelo.**
- Detección de anomalías: entradas fuera de distribución (posible evasión).

```
# validación con Pydantic + límite de tasa

from pydantic import BaseModel, Field

class Entrada(BaseModel):
    # rangos válidos = primera defensa
    edad: int = Field(ge=0, le=120)
    monto: float = Field(ge=0, le=1e6)

# rate limiting (p.ej. slowapi):
@limiter.limit("30/minute")
@app.post("/predict")
def predict(x: Entrada): ...
```

Capas 3–4 — Secretos, datos y modelo

Secretos y configuración

- **Credenciales FUERA del código: variables de entorno, .env, vault.**
- `.env` en `.gitignore`; versionar solo un `.env.example`.
- Rotación de claves y mínimo privilegio.
- TLS/HTTPS en tránsito; cifrado en reposo en el lake.

Datos y modelo (el lake)

- Acceso mínimo por bucket/prefijo (no 'todo o nada').
- **Versionado + integridad (checksum) del artefacto del modelo.**
- URLs prefiradas con expiración en vez de buckets públicos.
- Registro de acceso (quién leyó/escribió qué y cuándo).

Capa 5 — Entrenamiento robusto

Adversarial training

Entrenar con ejemplos perturbados para resistir ataques de evasión.

Privacidad diferencial

Ruido calibrado en el entrenamiento/updates para que el modelo no filtre datos.

Agregación robusta (federado)

Mediana / media recortada / Krum para tolerar clientes maliciosos (envenenamiento).

Validación de datos

Detectar y frenar datos envenenados antes de que entren al entrenamiento.

Estas defensas conectan con lo visto: la privacidad diferencial y la agregación robusta son del mundo del federado (S5); la validación de datos, del data lake (S6).

SAIF bajado a cada capa de la materia

Capa	Riesgo SAIF clave	Control a implementar (Mini-TP)
REST	Abuso, inyección, DoS, scraping	API key/JWT + validación Pydantic + rate limiting
GraphQL	DoS por consultas, fuga por introspección	Límite de profundidad/costo + authz por campo + sin introspección
gRPC	Acceso no autenticado, MITM	Interceptor de token en metadata + TLS + límite de tamaño
Streaming	Envenenamiento, productores no autorizados	Validación de esquema + cuarentena (DLQ) + autenticación
Federado	Cientes maliciosos, fuga por updates	Agregación robusta + clipping + privacidad diferencial
Data Lake	Exfiltración, acceso excesivo	Acceso mínimo + versionado/checksum + URLs prefiradas

Seguridad en REST y GraphQL

REST (FastAPI)

- Autenticar: API key o JWT como dependencia.
- Validar entrada con Pydantic (tipos, rangos, tamaño).
- Rate limiting por cliente: frena DoS y scraping.
- No filtrar trazas ni internals en los errores.
- HTTPS siempre; CORS restringido.

GraphQL

- **Límite de profundidad y de costo de la consulta (anti-DoS).**
- Desactivar introspección en producción (no filtrar el esquema).
- Autorización por campo/resolver, no solo al entrar.
- Validar argumentos; paginar listas; evitar consultas anidadas abusivas.
- Mismo backend de auth que REST.

Seguridad en gRPC y Streaming

gRPC

- Autenticación por token en la metadata (interceptor).
- TLS/mTLS: cifrar el canal y verificar extremos.
- Límite de tamaño de mensaje (anti-DoS).
- Autorización en el interceptor, antes del método.

Streaming (Kafka)

- Validar el esquema de cada evento: frena envenenamiento.
- Cuarentena a una dead-letter queue lo que no valida.
- Autenticar productores/consumidores (SASL) y ACLs por tópico.
- Registrar procedencia del evento (auditoría).

Seguridad en Federado y Data Lake

Aprendizaje federado

- **Agregación robusta (mediana/trimmed/Krum):** tolera clientes que envenenan.
- Clipping de la norma del update + detección de anomalías.
- Privacidad diferencial: evita que los updates filtren datos.
- Autenticar clientes; no confiar en cualquier participante.

Data Lake (MinIO/S3)

- Política de acceso mínima por bucket/prefijo.
- **Versionado + checksum:** integridad del dato y del modelo.
- URLs prefirmadas con expiración; nada de buckets públicos.
- Cifrado en reposo y registro de acceso (auditoría).

Demostración: ataque adversario

Una perturbación pequeña y deliberada en la entrada puede cambiar la predicción sin que un humano note la diferencia. Es el ataque de evasión.



Mitigaciones

- Adversarial training (entrenar con ejemplos perturbados).
- Detección de entradas fuera de distribución + rate limiting (el ataque requiere muchas consultas).
- Validación de rangos y suavizado; monitoreo de la tasa de cambio de predicciones.

Operación: mantener el modelo sano

Observabilidad

Logs, métricas y trazas del servicio y del modelo.

Monitoreo de drift

Vigilar cambios en datos y en la calidad de las predicciones.

SLAs y alertas

Objetivos de latencia/disponibilidad y aviso cuando se rompen.

Respuesta a incidentes

Plan para detectar, contener y recuperar (parte de SAIF).

Un modelo seguro que no se opera se degrada: el drift es una falla silenciosa que el monitoreo convierte en una alerta accionable.

Gobernanza: responsabilidad y confianza

Model cards

Documentar qué hace el modelo, con qué datos, límites y riesgos.

Auditoría / logging

Registro inmutable de accesos, predicciones y cambios.

Cumplimiento

Regulaciones (datos personales, sector); SAIF exige coherencia.

Ética y sesgo

Equidad, transparencia y explicabilidad (XAI) como confianza.

La model card y el logging de auditoría son la medida de gobernanza concreta que pide el Mini-TP 7.

Introducción a CI/CD para ML

La automatización (elemento 3 de SAIF) también aplica al despliegue: integrar y entregar cambios de modelo con pruebas, sin romper lo que funciona.



Una prueba de modelo que falla (métrica por debajo del mínimo, o un chequeo de sesgo) frena el despliegue, igual que un test roto frena el merge.

Caso de uso: detección de fraude con SAIF

El material base recorre un caso bancario de detección de fraude aplicando SAIF paso a paso. El mismo método sirve para el proyecto integrador.

Entender el uso objetivo, riesgos (falsos positivos, ataques adversarios)

Reunir el equipo perfiles diversos: IA, seguridad, negocio, legal

Nivelar conocimientos alinear a todos sobre qué es y qué no es la IA

Proteger cifrado, control de acceso, anonimización

Detectar y actuar monitoreo en tiempo real y respuesta a incidentes

Automatizar y mejorar defensas automáticas + reentrenar con fraude nuevo

Práctica: mapa de la sesión

Notebook integrador (teoría)

- Recorre las capas de seguridad sobre un servicio real.
- FastAPI con auth + validación + rate limiting + audit log.
- Demo de ataque adversario y su mitigación.
- Mapea cada control a un elemento de SAIF.

 `clase7/Teoria/seguridad_tutorial.ipynb`

 `clase7/Practica/mini_tp7_<protocolo>_actividad.ipynb`

Mini-TP 7 — elegir un protocolo

- Un notebook guiado por capa: REST, GraphQL, gRPC,
- streaming, federado y data lake.
- **El estudiante elige UNO y aplica las estrategias SAIF.**
- Cada uno corre de punta a punta (sin servicios externos).

Errores comunes

De seguridad

- Exponer el endpoint del modelo sin autenticación.
- Credenciales hardcodeadas o commiteadas por error.
- Confiar en la entrada del usuario sin validarla.
- Mensajes de error que filtran trazas e internals.
- 'La seguridad después': nunca llega.

De concepto

- Pensar que federado o cifrado = privado por sí solos.
- Creer que una sola capa alcanza (no hay bala de plata).
- Ignorar el drift: el modelo se degrada en silencio.
- Seguridad sin gobernanza: sin auditoría no hay rendición de cuentas.
- No medir: sin logs no se detecta ni se responde.

Mini-TP 7 — aplicar SAIF a un protocolo

Consigna (individual)

- **Elegir UNA capa:** REST, GraphQL, gRPC, streaming, federado o data lake.
- Abrir su notebook guiado y aplicar las estrategias SAIF de esa capa.
- Completar los TODO: el control de seguridad principal sobre el modelo propio.
- Agregar una medida de gobernanza: model card o logging de auditoría.
- Reflexión: qué amenaza se mitiga y a qué elemento de SAIF corresponde.

Cierra el ciclo de Mini-TPs

- **Es el último Mini-TP individual (Sesiones 1–7).**
- Al terminarlo, tu modelo ya está: servido por REST/GraphQL/gRPC,
- integrado a streaming, entrenable en federado, guardado en un
- data lake y ahora asegurado.
- Todo listo para el Taller integrador y la defensa (Sesión 8).

Cierre y qué sigue

Lo que queda

- La seguridad es transversal: de los datos al despliegue, la operación y la gobernanza.
- SAIF da el marco; la defensa en profundidad lo vuelve capas concretas de controles.
- **Cada protocolo visto tiene su riesgo propio y su control: ese es el Mini-TP 7.**
- Operar (monitoreo, drift) y gobernar (model cards, auditoría) son parte de la seguridad.

Próxima sesión → Taller integrador y defensa: se juntan todas las capas en el proyecto final.

REST

GraphQL

gRPC

Streaming

Federado

Cloud/Lake

Seguridad

Integra

Referencias

- Google SAIF — Secure AI Framework: safety.google/cybersecurity-advancements/saif/
- Google Cloud Skills Boost — ruta de seguridad en IA (material base del curso)
- OWASP — Top 10 para LLM/ML y ML Security Top 10
- NIST AI Risk Management Framework (AI RMF)
- MITRE ATLAS — tácticas y técnicas adversarias contra sistemas de IA
- Model Cards (Mitchell et al.) · Papers de ataques adversarios (Goodfellow et al.)
- **Repositorio del curso — [clase7/Teoria/](#) y [clase7/Practica/](#)**